

整合練習 1：Function Module vs BAPI

Lecture

基礎課 ex15 已經教過 **Function Module (FM)**：一段掛在 Function Group 底下、有明確 **IMPORTING/EXPORTING/TABLES/EXCEPTIONS** 介面的可重用邏輯，用 SE37 建立與測試，用 **CALL FUNCTION** 呼叫。REST 課 rs10 則已經實際呼叫過一支叫 **BAPI_FLBOOKING_CREATEFROMDATA** 的東西——語法上看起來就是 **CALL FUNCTION 'xxx'**，跟呼叫一般 FM 完全一樣。這題要講清楚：**BAPI 其實就是 FM**，但是「特殊的」**FM**，特殊在哪裡、為什麼要有這層區分。

BAPI (Business API) 的本質：是一支同時滿足下面幾個條件的 Function Module——

1. 命名慣例固定：**BAPI_<Business Object>_<動作>**（例如 **BAPI_FLBOOKING_CREATEFROMDATA** 對應「訂位」這個 Business Object 的「CreateFromData」動作），不像一般 Z FM 名稱自訂
2. 一定是 **Remote-Enabled Module (RFC-enabled)**：這是 BAPI 存在的目的——設計上就是要給「外部系統」或「其他 SAP 系統」透過 RFC 呼叫，不是只給同一顆程式內部用。一般 FM 預設不是 Remote-Enabled，除非開發者自己在 SE37 的 Attributes 頁籤手動勾選
3. 掛在 **BOR (Business Object Repository)** 底下，是某個 Business Object（用 SWO1 可以瀏覽到）的一個 Method——這一層「掛勾」讓 BAPI 可以被 BAPI Explorer、其他建模工具、甚至第三方系統的 Business Object 導覽介面找到，一般 FM 沒有這層登記
4. 有 **SAP 官方的介面相容承諾 (Release 承諾)**：BAPI 一旦發布，SAP 保證不會隨意修改既有參數的意義（頂多用「新增可選參數」的方式擴充），因為外部系統可能已經依賴這個介面在跑——這跟一般 Z FM 完全相反：Z FM 的介面是團隊自己的資產，想改就改，只要通知呼叫端配合修改即可
5. 錯誤回報用結構化的 **RETURN** 表格（型別通常是 **BAPIRET2**，欄位包含 **TYPE (E/W/S/I/A)**、**ID**、**NUMBER**、**MESSAGE**），呼叫端自己檢查這張表判斷成敗；不像一般 FM 常用簡單的 **EXCEPTIONS (sy-subrc <> 0** 就知道出錯，但錯誤細節要另外查)
6. 寫入資料庫的 **BAPI 一律不做 COMMIT WORK**：LUW 邊界留給呼叫端決定（這個規則本身在 if03 會深入講；這題只需要先認出「這是 BAPI 的一個特徵」）

對照練習用的兩個真實物件（都是本課程之前已經用過、確實存在、已驗證能跑的東西，不是憑空舉例）：

- 一般 FM 對照組：ex15 建立的 **Z_TR15_CALC_REVENUE**——純運算（票價 × 已售座位 = 營收），沒有任何資料庫寫入，**IMPORTING/EXPORTING** + 一個 **EXCEPTIONS invalid_input**
- **BAPI 對照組**：rs10 用過的 **BAPI_FLBOOKING_CREATEFROMDATA**（建立訂位）+ **BAPI_TRANSACTION_COMMIT**（提交異動），呼叫端程式碼是 **zcl_rs10_booking_batch_service.clas.abap**

學習目標

- 能講出 BAPI 跟一般 Function Module 的本質差異，至少涵蓋：命名慣例 / 是否 Remote-Enabled / 是否掛在 BOR / 有沒有 SAP 官方相容承諾 / 錯誤回報方式（**RETURN** 結構化 vs **EXCEPTIONS**）/ **COMMIT** 責任歸屬
- 能用 SE37 的 Attributes 頁籤判斷一支 FM 是不是 Remote-Enabled Module
- 能用 BAPI Explorer（交易碼 **BAPI**）或 **SWO1** 找到一支 BAPI 掛在哪個 Business Object 底下
- 能對照既有程式碼，指出「結構化錯誤回報」與「呼叫端自己 COMMIT」這兩段分別對應哪些程式碼

事前準備

不需要新建任何 SAP 物件——這題是「比較與觀察」，直接沿用兩個已經存在、已經驗收過的物件：

- `Z_TR15_CALC_REVENUE` (基礎課 ex15，一般 FM 對照組，快照見 `src/ABAP_Training/z_tr15_calc_revenue.func.abap`)
- `BAPI_FLBOOKING_CREATEFROMDATA` + `BAPI_TRANSACTION_COMMIT` (SAP 標準 BAPI，rs10 已經呼叫過，呼叫端程式快照見 `src/ABAP_Training_REST/zcl_rs10_booking_batch_service.clas.abap`)

題目需求

1. 用 **SE37** 分別開啟兩支 FM，確認下面比較表的 **Processing Type / Release** 狀態 (已用 `sap-adt` MCP 連線本系統 client 130 實際查證，SE37 Attributes 頁籤看到的內容會跟這裡一致)：

| 面向 | `Z_TR15_CALC_REVENUE` (一般 FM) | `BAPI_FLBOOKING_CREATEFROMDATA` (BAPI) | |---|---|---| | 命名慣例 | 自訂 (`Z_` 前綴 + 自由描述) | 固定 `BAPI_<物件>_<動作>` | | 是否 Remote-Enabled | 否——`processingType="normal"` (查證得到) | 是——`processingType="rfc"` | | Release 狀態 | 無此概念 (`Z` 物件不會有 Release 狀態) | `releaseState="external" + releaseDate="2001-02-08"`——SAP 官方在 2001 年就把這支介面正式對外發布，這是「相容承諾」最直接的證據，不是憑空推論 | | 是否掛在 BOR | 否，沒有對應 Business Object | 這格查不到——傳統 BOR (Business Object Repository，SWO1 管理) 目前的 ADT `discovery` 只找到 BOPF (`/sap/bc/adt/bopf/businessobjects`，較新的 RAP 世代框架) 的 collection，沒有找到 SWO1 / 傳統 BOR 的 ADT API；這一格屬於本專案已知的「GUI-only」類別 (跟 `.claude/rules/sap-adt-mcp.md` 記載的 Search Help / T-code 同一種限制)，要拿到答案得靠 SWO1 或 BAPI Explorer 交易碼手動查 | | 錯誤回報方式 | `EXCEPTIONS invalid_input` (`sy-subrc <> 0` 判斷) | `TABLES return TYPE bapiret2` (逐筆檢查 `type = 'E'/'A'`) | | 資料庫寫入 | 無 (純運算) | 有 (建立訂位資料) | | 是否需要呼叫端自己 COMMIT | 不適用 (沒有寫入) | 是，`BAPI_TRANSACTION_COMMIT` | | SAP 官方相容承諾 | 無 (團隊自己的 `Z` 物件，介面可自由調整) | 有 (見上方 Release 狀態欄，`releaseState="external"` 就是這個承諾的技術落地) |

1. 對照 `zcl_rs10_booking_batch_service.clas.abap` 的 `create_bookings` 方法，指出下面兩段程式碼分別對應 Lecture 提到的哪個 BAPI 特徵：

```
``abap " 片段 A LOOP AT lt_return INTO DATA(ls_msg) WHERE type = 'E' OR type = 'A'.
lv_error_message = ls_msg-message.EXIT. ENDLOOP. ``
```

```
``abap " 片段 B CALL FUNCTION 'BAPI_TRANSACTION_COMMIT' EXPORTING wait = abap_true. ``
```

(片段 A 對應「結構化錯誤回報」——`lt_return` 是 `BAPIRET2` 表，用 `type` 欄位判斷是不是錯誤 / 中斷等級的訊息；片段 B 對應「BAPI 本身不做 COMMIT WORK，呼叫端要自己呼叫 `BAPI_TRANSACTION_COMMIT`」)

1. 思考分析 (只需要寫分析，不用真的動手改程式)：如果要把 `Z_TR15_CALC_REVENUE` 改造成一支可以給外部系統呼叫的 BAPI，需要做哪些改造？至少列出四項，對照 Lecture 的六個 BAPI 條件逐一檢視。

參考答案 (思考分析)

把 `Z_TR15_CALC_REVENUE` 改造成 BAPI，至少要做：

1. 改名：Z_TR15_CALC_REVENUE → 類似 BAPI_FLIGHTREVENUE_CALCULATE 這種 BAPI_<物件>_<動作> 格式（但這個運算本身沒有對應到任何 Business Object，這一步其實會卡住——這正好呼應下面思考題 1）
2. 勾選 **Remote-Enabled Module**：SE37 Attributes 頁籤把 Processing Type 改成 Remote-Enabled，否則外部系統 / RFC 呼叫端根本連不到
3. 掛到 **BOR**：要嘛用 SW01 建一個新的 Business Object 把這個 Method 掛上去，要嘛找一個既有的、語意相符的 Business Object 掛過去——但「計算營收」是一段純運算，不對應任何具體的業務實體（不像訂位、庫存這種有明確主檔的東西），這一步在真實情境下通常代表「這段邏輯根本不該做成 BAPI」
4. 把 **EXCEPTIONS invalid_input** 改成 **TABLES return TYPE STANDARD TABLE OF bapiret2**：iv_price < 0 這種輸入驗證失敗，改成往 return 表塞一筆 TYPE = 'E' 的訊息，而不是丟例外
5. 承擔 **SAP（或團隊內部）的介面相容承諾**：一旦公開，iv_price / iv_seatsocc 這兩個參數的型別、意義都不能再隨便改，要維護成本

思考題

1. 上面第 3 點刻意卡住的地方——Z_TR15_CALC_REVENUE 是純運算，沒有對應任何 **Business Object**，「掛到 **BOR**」這一步做不下去——這說明了什麼？（提示：BAPI 不是「介面設計得比較講究的 FM」，而是專門用來代表「對某個業務實體做某件事」，純函式運算不是 BAPI 該解決的問題，應該就留在一般 FM 或 Class Method 層級）
2. BAPI_FLBOOKING_CREATEFROMDATA 屬於 SAP 標準示範套件 SAPBC_IBF_SBOOK。如果今天在正式的 Z 專案裡要做一個「自訂訂單建立」功能，什麼條件下才值得做成 BAPI？什麼條件下該用一般 Class Method 或 Z FM 就好？（提示：只有「真的要開放給外部系統 / 其他模組透過 RFC 呼叫」且「願意承擔長期介面穩定的維護成本」時才做成 BAPI；純內部使用、還在快速迭代的邏輯，用 Class Method 更靈活，之後真的要對外開放時再考慮包一層 BAPI）
3. Z_TR15_CALC_REVENUE 目前不是 Remote-Enabled Module（可在 SE37 確認），但它其實可以被「勾選成」Remote-Enabled 而完全不改名、不掛 BOR。這樣它會變成 BAPI 嗎？為什麼？（提示：Remote-Enabled 只是 BAPI 的必要條件之一，不是充分條件——三個條件缺一都不算 BAPI，這也是很多人誤以為「RFC-enabled 的 FM 就是 BAPI」的常見迷思）

答案

比較表與程式碼對照見本題內文；不新建 SAP 物件，沿用基礎課 ex15 Z_TR15_CALC_REVENUE（快照 src/ABAP_Training/z_tr15_calc_revenue.func.abap）與 REST 課 rs10 BAPI_FLBOOKING_CREATEFROMDATA 呼叫端 zcl_rs10_booking_batch_service.clas.abap（快照 src/ABAP_Training_REST/）。比較表已用 sap-adt MCP 連線本系統 client 130 實際查證：Z_TR15_CALC_REVENUE 的 processingType="normal"（非 Remote-Enabled）；BAPI_FLBOOKING_CREATEFROMDATA 的 processingType="rfc"、releaseState="external"、releaseDate="2001-02-08"（Function Group SAPBC_BAPI_SBOOK，套件 SAPBC_IBF_SBOOK）。唯一沒有查到的是「BAPI 掛在哪個 BOR Business Object」——這格經確認屬於本專案已知的 GUI-only 限制（ADT 沒有傳統 BOR / SWO1 的 API），留待有 SAP GUI 的人用 SWO1 或 BAPI Explorer 補上，不影響本題其餘結論。